

OWLBOX

Verkabelung

Vollständige Hardware-Referenz:
Pinbelegung, Display-Treiber, Backlight, Netzwerk-Fallback.

Hardware-Aufbau Raspberry Pi 3B+
Schnelleinstieg: OwlBox-Schnellstart.pdf

Inhalt

Inhalt	2
1. Zielhardware	3
Physische Steckplatz-Kollision: Display vs. HiFiBerry	3
GPIO-Belegung im Überblick	4
2. HiFiBerry Amp2	6
Lautsprecher anschließen	6
3. RC522 RFID-Leser (Software-SPI auf freien GPIOs)	7
4. Taster (vor/zurück)	8
5. Dreh-Encoder mit Taster (KY-040)	9
Zweiter Dreh-Encoder für Helligkeit (KY-040)	9
6. Dimmbares Display-Backlight (Pflicht)	10
7. Fallback-Hotspot (WLAN-Recovery)	11
8. 3,5" SPI-Display: Treiber (ohne Touch)	12
Wichtig: legacy Grafiktreiber statt Wayland/labwc	12
9. Kiosk-Autostart (kein Desktop)	13
10. Konfigurationsdatei (config.yaml)	14

1. Zielhardware

- Raspberry Pi 3B+
- HiFiBerry Amp2 (I2S-Verstärker-HAT, TAS5756M-Chip)
- RC522 RFID-Modul (SPI, 13,56 MHz)
- 3,5" SPI-Touchscreen, 480×320, 26-Pin-Header - sehr wahrscheinlich ein „MHS-35“/„tft35a“-Klon (ILI9486 + XPT2046, unter vielen Markennamen identisch verkauft). Touch bleibt bewusst deaktiviert.
- 2 Taster (vor/zurück)
- 1 Dreh-Encoder mit Druckschalter (Lautstärke/Play-Pause)
- 1 weiterer Dreh-Encoder ohne Taster (Helligkeit)
- 1 NPN-Transistor (z.B. BC547) oder Logic-Level-N-MOSFET für dimmbares Backlight

Alle Pin-Angaben sind BCM-Nummerierung und entsprechen den Standardwerten in config/config.example.yaml. Wer andere Pins verdrahtet, passt einfach die gpio:/rfid:-Sektion in config.yaml an.

Hinweis: Referenzgrafiken liegen zusätzlich zu dieser PDF im Projekt unter docs/: owlbox-wiring-diagram.svg (Gesamtaufbau), owlbox-gpio-pinout.svg (kompletter 40-Pin-Header), owlbox-adapter-layout.svg (Platinenlayout-Vorschlag) und hat-wiring.html (kompletter Schaltplan, im Browser öffnen).

Physische Steckplatz-Kollision: Display vs. HiFiBerry

Das Display soll laut Beschreibung direkt auf den GPIO-Header gesteckt werden - der HiFiBerry braucht aber ebenfalls einen direkten Sitz auf demselben Header (I2S ist empfindlich gegenüber zusätzlichen Steckverbindern). **Beide gleichzeitig aufstecken geht nicht.**

Lösung: Das Display nicht aufstecken, sondern per Jumper-/Dupont-Kabeln verdrahten. Der HiFiBerry sitzt normal direkt auf dem Pi, das Display hängt per Kabel daneben - und es müssen nur die tatsächlich gebrauchten Leitungen verbunden werden. Die Touch-Leitungen (CE1/PENIRQ) werden dabei einfach gar nicht erst angeschlossen, Touch bleibt so elektrisch inaktiv.

Hinweis: Der Display-Treiber (mhs35/tft35a-Overlay) meldet dem Kernel trotzdem einen Touch-Controller auf SPI0 CE1 an - fest im Overlay einprogrammiert, ohne Parameter zum Abschalten, unabhängig davon, ob Touch physisch angeschlossen ist. Beide SPI0-Chipselects sind damit softwareseitig belegt, der RC522 kann nicht mit auf SPI0 - siehe Kapitel 3 für die tatsächliche Verkabelung über Software-SPI auf freien GPIOs.

Display-Pin (26-Pin-Header)	Pi-Pin (BCM)	Zweck
VCC	3.3V	Versorgung
GND	GND	Masse
SCK	GPIO11	SPI0-Takt (nur Display, RC522 hängt an eigenen GPIOs, s. Kapitel 3)

Display-Pin (26-Pin-Header)	Pi-Pin (BCM)	Zweck
MOSI (SDI)	GPIO10	SPI0 (nur Display)
MISO (SDO)	GPIO9	SPI0 (nur Display)
CS/CE0	GPIO8	Display-Chipselect
DC/RS	GPIO24	Data/Command (Standardwert des tft35a-Overlays)
RST	GPIO25	Reset (Standardwert des tft35a-Overlays)
LED/Backlight	GPIO13, über Treibertransistor	dimmbar, siehe Kapitel 3
T_CLK, T_CS, T_DIN, T_DO, T_IRQ (Touch)	nicht anschließen	Touch bleibt so auch elektrisch inaktiv

GPIO-Belegung im Überblick

Funktion	BCM-Pin	Genutzt von
I2S BCLK	18	HiFiBerry (Display-Backlight bewusst NICHT hierauf gelegt)
I2S LRCLK	19	HiFiBerry
I2S DIN	20	HiFiBerry
I2S DOUT	21	HiFiBerry
I2C SDA	2	HiFiBerry (Amp-Steuerung)
I2C SCL	3	HiFiBerry (Amp-Steuerung)
SPI0 SCLK/MOSI/MISO	11 / 10 / 9	Display (RC522 hängt NICHT hier, s. Kapitel 3)
SPI0 CE0	8	Display (TFT-Chipselect)
SPI0 CE1	7	vom Display-Overlay softwareseitig für Touch reserviert - unbenutzbar
Display DC	24	Display
Display RST	25	Display
RC522 SCK (Software-SPI)	4	RC522 (rfid.sck_pin)
RC522 MOSI (Software-SPI)	16	RC522 (rfid.mosi_pin)
RC522 MISO (Software-SPI)	15	RC522 (rfid.miso_pin)
RC522 SDA/CS (Software-SPI)	14	RC522 (rfid.cs_pin)
RC522 RST	26	RC522 (rfid.reset_pin)
Taster Weiter	5	Taster

Funktion	BCM-Pin	Genutzt von
Taster Zurück	6	Taster
Encoder CLK	1	Lautstärke-Encoder (NICHT 17, s. Kapitel 3)
Encoder DT	27	Lautstärke-Encoder
Encoder SW	22	Lautstärke-Encoder
Display-Backlight (dimmbar)	13	Backlight-Dimmen (Treibertransistor)
Helligkeits-Encoder CLK	23	Helligkeits-Encoder
Helligkeits-Encoder DT	12	Helligkeits-Encoder

2. HiFiBerry Amp2

Der HiFiBerry belegt die I2S-Pins (BCM 18/19/20/21) sowie I2C (BCM 2/3) zur Verstärkersteuerung. In /boot/firmware/config.txt (bzw. /boot/config.txt auf älteren Images):

```
dtparam=audio=off
dtoverlay=hifiberry-dacplus
```

Hinweis: An echter Hardware bestätigt: Der Amp2 hat einen TAS5756M-Chip - dieselbe PCM512x-Chipfamilie wie die DAC+ Pro, ein komplett anderer Chip als der TAS5713 des älteren Amp/Amp+. `dtoverlay=hifiberry-amp` ist speziell für den TAS5713 und funktioniert mit dem Amp2 nicht: der Kernel spricht dann die falsche I2C-Adresse an, `aplay -l` zeigt „no soundcards found“ - äußert sich als Lautstärke, die sich nie ändert (bleibt bei 0). Zum Nachprüfen, welcher Chip verbaut ist: `i2cdetect -y 1` - Adresse 0x4d antwortet beim TAS5756M/Amp2 (hifiberry-dacplus), Adresse 0x1b beim TAS5713 vom Amp/Amp+ (hifiberry-amp).

Nach einer Overlay-Änderung neu starten - dabei verschiebt sich meist auch die Kartennummer.

Danach mit `aplay -l` die Kartennummer der HiFiBerry ermitteln (z.B. „card 2: ...“) und mit `amixer -c <Kartennummer> scontrols` den Mixer-Namen prüfen - beides in `config.yaml` eintragen: `audio.alsa_device ("hw:<Kartennummer>,0")`, `audio.mixer_control` (Amp/Amp2 nutzen meist „Digital“, manche Boards „PCM“ oder „Master“) **und** `audio.mixer_card` (nur die Kartennummer, ohne `hw:/,0`).

Wichtig: Alle drei müssen zur selben Karte passen - `install.sh` trägt sie bei der automatischen Erkennung mittlerweile alle drei ein, aber wer das von Hand einträgt, vergisst leicht `mixer_card`: bleibt die auf ihrem Standardwert "0" stehen während die HiFiBerry tatsächlich auf einer anderen Kartennummer läuft, zielt jede Lautstärkeabfrage/-änderung ins Leere - äußert sich als „eingestellte Lautstärke wird nie gespeichert, zeigt immer 0“.

Lautsprecher anschließen

Der HiFiBerry Amp2 hat dafür keine Stecker (kein Cinch/Klinke), sondern zwei 2-polige Federklemmen direkt auf der Platine (eine pro Kanal, jeweils +/-). Angeschlossen wird ganz normales 2-adriges Lautsprecherkabel:

- Querschnitt $\geq 0,75 \text{ mm}^2$ (AWG 18) reicht für 15W/4Ω locker; bei längeren Kabelwegen (> 3-5m) eher 1,0-1,5 mm^2 nehmen.
- Enden abisolieren (~10mm); bei feindrähtiger Litze verzinnen oder Aderendhülsen verwenden, damit die Federklemme sauber greift.
- Polarität an beiden Lautsprechern konsistent anschließen (+ zu + und Minus zu Minus) - sonst laufen sie gegenphasig und Bass/Stereo-Ortung leiden.
- Am Lautsprecher selbst hängt der Anschluss vom jeweiligen Modell ab (blanker Draht, Flachsteckhülsen/Bananas oder Lötflansen).

3. RC522 RFID-Leser (Software-SPI auf freien GPIOs)

Hinweis: Anders als in den meisten RC522-Anleitungen im Netz hängt der RC522 hier NICHT an einem der beiden Hardware-SPI-Busse des Pi, sondern an vier per Software angesteuerten GPIOs. Grund: SPI0 ist komplett vom Display-Treiber belegt (CE0 fürs Display, CE1 fest für einen Touch-Controller reserviert, siehe Kapitel 1), SPI1 liegt auf GPIO18-21 - exakt den Pins, die der HiFiBerry für I2S-Ton braucht. Der RC522 hat keine Mindesttaktrate, Software-SPI funktioniert daher zuverlässig, nur etwas langsamer als Hardware-SPI - für einen Chip-Scan völlig ausreichend.

RC522-Pin	Raspberry Pi	Config-Feld
VCC	3.3V (nicht 5V!)	-
GND	GND	-
RST	GPIO26	rfid.reset_pin
SDA (CS)	GPIO14	rfid.cs_pin
SCK	GPIO4	rfid.sck_pin
MOSI	GPIO16	rfid.mosi_pin
MISO	GPIO15	rfid.miso_pin
IRQ	nicht verbunden	-

Alle vier GPIOs (4/14/15/16) sind sonst von nichts in diesem Projekt belegt. SPI selbst muss trotzdem aktiviert bleiben, weil das Display es braucht (macht scripts/install.sh bereits via raspi-config nonint do_spi 0, alternativ sudo raspi-config → Interface Options → SPI).

Hinweis: An echter Hardware bestätigt: Der Display-Treiber beansprucht zusätzlich zu SPI0 auch noch GPIO17 als Interrupt-Pin („pendown“) für den (nie verdrahteten) Touch-Controller - unabhängig davon, ob Touch angeschlossen ist. Zwischen Display-Overlay und diesem Projekt ist dadurch jeder GPIO von 2-27 belegt. Der Lautstärke-Encoder liegt deshalb auf GPIO1 statt dem naheliegenderen GPIO17 - siehe Kapitel 5.

4. Taster (vor/zurück)

Als Taster kommen Cherry-MX-Switches (3-Pin-Variante) zum Einsatz - elektrisch ganz normale Momentary-Schalter (schließt nur beim Drücken, öffnet sonst).

- Von den 3 Pins sind nur die beiden Metall-Pins die elektrischen Kontakte (diagonal gegenüberliegend); der dritte, meist aus Kunststoff, ist nur ein mechanischer Halteclip ohne elektrische Funktion und muss nirgends angeschlossen werden.
- Jeweils einer der beiden Metall-Pins an GPIO, der andere an GND. Kein externer Widerstand nötig, der interne Pull-up wird von gpiozero aktiviert.
- Da Cherry-MX-Switches (anders als billige Blechtaster) sehr sauber prellen, reicht die Standard-Entprellzeit (`gpio.bounce_time`, 50ms) komfortabel aus.

Funktion	BCM-Pin
Zurück	6
Weiter	5

Kurz drücken springt zum vorherigen/nächsten Track. Gedrückt halten (länger als `gpio.seek_hold_seconds`, Standard 0,4s) spult stattdessen im aktuellen Track vor/zurück, in Schritten von `gpio.seek_step_seconds` (Standard 10s) - kein Trackwechsel, solange gehalten wird.

5. Dreh-Encoder mit Taster (KY-040)

Encoder-Pin	Raspberry Pi
CLK	GPIO1 (nicht 17, siehe Kapitel 3)
DT	GPIO27
SW	GPIO22
+	3.3V
GND	GND

Das KY-040-Modul bringt eigene Pull-up-Widerstände für CLK/DT mit; die zusätzlich von gpiozero aktivierten Pull-ups des Pi stören dabei nicht (einfach parallel). Für den Taster (SW) hat das Modul in der Regel keinen eigenen Pull-up - das übernimmt `gpiozero.Button(pull_up=True)` im Code, hier also nichts weiter nötig.

Drehen ändert die Lautstärke (Schrittweite `audio.volume_step`), Drücken schaltet Play/Pause um. Ein langer Druck (`gpio.shutdown_hold_seconds`, Standard 4s) fährt den Pi sicher herunter - praktisch für ein Kindergerät ohne Zugriff auf ein Terminal. Auf 0 setzen, um das abzuschalten.

Zweiter Dreh-Encoder für Helligkeit (KY-040)

Gehört zum Standardaufbau, zusammen mit dem dimmbaren Backlight (Kapitel 6) - dasselbe KY-040-Modul, diesmal ohne den Taster zu verdrahten (kein eigener Klick, nur Drehen).

Encoder-Pin	Raspberry Pi
CLK	GPIO23
DT	GPIO12
+	3.3V
GND	GND

Drehen ändert die Helligkeit (Schrittweite `gpio.brightness_step`, Standard 5%), sofort und rein manuell - es gibt kein automatisches Dimmen.

Hinweis: Damit Shutdown/Neustart (auch über die Web-UI oder einen Funktions-Chip) sowie der Update-Button auf der Info-Seite ohne Passwortabfrage funktionieren, braucht der Service-User owlbox passwortloses sudo dafür. `install.sh` richtet das automatisch ein (`/etc/sudoers.d/owlbox`, syntaxgeprüft vor dem Einspielen) - hier nur zur Referenz:

```
owlbox ALL=(ALL) NOPASSWD: /sbin/shutdown, /usr/bin/nmcli, \
/usr/bin/systemctl restart --no-block owlbox
```

Wichtig: Die Argumente müssen exakt wie oben dastehen (inklusive `--no-block`) - sudo vergleicht die komplette Befehlszeile. Fehlt `--no-block`, meldet der Update-Button auf der Info-Seite beim Neustart „sudo: a password is required“.

6. Dimmbares Display-Backlight (Pflicht)

Je nach Fertigungscharge ist die LED-Leitung bei diesem Board-Typ ab Werk entweder fest verdrahtet oder auf einen GPIO gelegt (öfter berichtet: GPIO18 - genau der Pin, den der HiFiBerry für die I2S-Bit-Clock braucht, hier also nicht verwendbar). Bei OwlBox ist das Backlight-Dimmen **Pflicht, kein optionales Extra** - der Helligkeitsregler in den Einstellungen und der zweite Dreh-Encoder sind zentrale Bedienelemente.

- Die LED-Leitung nicht an 3.3V, sondern an einen freien GPIO anschließen - GPIO13 (physischer Pin 33, einer der Hardware-PWM-fähigen Pins neben 12/18/19, von denen 18/19 dem HiFiBerry gehören).
- Da ein Pi-GPIO nicht genug Strom für die Hintergrundbeleuchtung liefern kann, einen kleinen NPN-Transistor (z.B. BC547) oder Logic-Level-N-MOSFET als Schalter dazwischenschalten: GPIO13 → Basis/Gate (über $\sim 1\text{k}\Omega$ Vorwiderstand bei einem BJT), Kollektor/Drain → LED-Kathode, Emitter/Source → GND. Die LED-Anode bleibt wie gehabt an 3.3V bzw. an der vom Board vorgesehenen Versorgung.
- In config.yaml: gpio.backlight_pin: 13 setzen (Standard in config.example.yaml). Nur auf null setzen, wenn das Backlight abweichend vom Standardaufbau doch fest an 3.3V hängt - dann hat der Regler in den Einstellungen keine Wirkung.
- Den zweiten KY-040-Dreh-Encoder (Kapitel 5) für die Helligkeit verdrahten - Drehen ändert die Helligkeit sofort um gpio.brightness_step (Standard 5%) pro Rastung.

7. Fallback-Hotspot (WLAN-Recovery)

Ist WLAN eingeschaltet, aber für `network.hotspot_after_seconds` (Standard 60s) mit keinem Netzwerk verbunden - z.B. weil das Heimnetz sein Passwort geändert hat oder der Pi an einen neuen Ort umgezogen ist - macht der Pi automatisch seinen eigenen Access Point auf (nmcli device wifi hotspot), statt komplett unerreichbar zu bleiben.

SSID und Passwort (aus `config.yaml` unter `network:`, Standard „OwlBox-Setup“ / „owlbox-setup“) werden auf dem Kiosk-Display und im Admin-Bereich angezeigt, inklusive der URL, unter der die Einstellungen im Hotspot erreichbar sind (normalerweise `http://10.42.0.1:5000/admin` - NetworkManagers Standard-Adresse für einen geteilten Access Point).

Ablauf: mit einem Laptop/Handy in dieses WLAN einwählen, die angezeigte URL öffnen, unter Einstellungen → WLAN das eigentliche Netzwerk auswählen/verbinden. Sobald das klappt, beendet der Pi den Hotspot von selbst wieder.

Wichtig: Das Standardpasswort „owlbox-setup“ steht so im Quellcode und ist damit öffentlich bekannt - für den Einsatz in einer Umgebung, in der Fremde in Funkreichweite kommen könnten, unbedingt in `config.yaml` ein eigenes Passwort setzen.

8. 3,5" SPI-Display: Treiber (ohne Touch)

Diese Board-Familie (tft35a/MHS-35) funktioniert nicht über einen direkten KMS-Grafikausgang, sondern über einen Trick: der Pi bekommt per config.txt einen „unsichtbaren“ virtuellen HDMI-Ausgang in der Auflösung des Displays vorgegaukelt, X11/Chromium rendern ganz normal dorthin, und ein kleines Hilfsprogramm (fbcp) kopiert das Bild laufend per SPI aufs eigentliche TFT.

Empfohlener Weg: statt config.txt von Hand zu editieren, den vom Verkäufer verlinkten Treiber-Installer benutzen, oder alternativ das quelloffene goodtft/LCD-show-Skript (auf GitHub, Skriptname i.d.R. MHS35-show oder LCD35-show) - der Installer setzt automatisch die passenden config.txt-Werte, kompiliert/installiert fbcp und richtet den Autostart ein.

Danach nur einen Schritt selbst nachziehen: in /boot/firmware/config.txt die vom Installer eingetragene Touch-Zeile wieder entfernen/auskommentieren - sie beginnt mit dtoverlay=ads7846,... . Ohne diese Zeile lädt der Kernel den XPT2046-Touch-Treiber gar nicht erst, Touch ist damit auch softwareseitig aus.

Zur Referenz, wie diese Zeilen ungefähr aussehen (der Installer setzt sie automatisch):

```
hdmi_force_hotplug=1
hdmi_group=2
hdmi_mode=87
hdmi_cvt=480 320 60 6 0 0 0
hdmi_drive=2
dtparam=spi=on
dtoverlay=tft35a:rotate=90
```

Und als systemd-Service für fbcp, falls der Installer keinen eigenen Autostart einrichtet (Vorlage liegt in systemd/owlbox-fbcp.service):

```
sudo cp systemd/owlbox-fbcp.service /etc/systemd/system/
sudo systemctl daemon-reload
sudo systemctl enable --now owlbox-fbcp.service
```

Wichtig: legacy Grafiktreiber statt Wayland/labwc

fbcp liest von /dev/fb0 - das gibt es unter dem modernen KMS-Grafiktreiber (vc4-kms-v3d, Standard seit Raspberry Pi OS Bullseye/Bookworm mit Wayland/labwc) meist nicht mehr in nutzbarer Form. Für diese Display-Familie also in /boot/firmware/config.txt den KMS-Treiber deaktivieren:

```
#dtoverlay=vc4-kms-v3d
```

und über sudo raspi-config → Advanced Options → GL Driver auf „Legacy“ stellen. Empfohlenes Basis-Image ist deshalb Raspberry Pi OS (Legacy) Lite, 64-bit - der alte Grafiktreiber, aber bewusst ohne mitinstallierte Desktop-Umgebung (siehe Kapitel 9, warum).

9. Kiosk-Autostart (kein Desktop)

scripts/kiosk.sh startet Chromium im Kiosk-Modus gegen <http://localhost:5000/> - das funktioniert, sobald fbcp läuft und der legacy Grafiktreiber (nicht Wayland) aktiv ist. Auf „Legacy Lite“ gibt es aber keine Desktop-Umgebung (kein lightdm, kein LXDE), in die sich der Kiosk einhängen könnte - deshalb startet ein eigener systemd-Dienst (owlbox-kiosk.service) X direkt selbst per startx, übernimmt dafür tty1 und lässt scripts/kiosk.sh als einzigen X-Client laufen. Kein Panel, kein Dateimanager-Desktop, kein Login-Bildschirm - das spart gegenüber einer vollen Desktop-Umgebung spürbar Bootzeit.

```
sudo apt-get install -y xserver-xorg-video-fbdev

# X ohne Display-Manager erlauben:
cat > /etc/X11/Xwrapper.config <<'EOF'
allowed_users=anybody
needs_root_rights=yes
EOF

# Mit dem Legacy-GL-Treiber (kein /dev/dri/card0) scheitert X's
# automatisch gewählter modesetting-Treiber mit 'no screens found' -
# stattdessen ausdrücklich auf den Framebuffer zeichnen lassen:
mkdir -p /etc/X11/xorg.conf.d
cat > /etc/X11/xorg.conf.d/99-owlbox-fbdev.conf <<'EOF'
Section "Device"
    Identifier "OwlBoxFramebuffer"
    Driver "fbdev"
    Option "fbdev" "/dev/fb0"
EndSection

Section "Screen"
    Identifier "OwlBoxScreen"
    Device "OwlBoxFramebuffer"
EndSection
EOF

sudo cp /opt/owlbox/systemd/owlbox-kiosk.service /etc/systemd/system/
sudo systemctl daemon-reload
sudo systemctl enable --now owlbox-kiosk.service
```

Hinweis: owlbox-kiosk.service bringt Conflicts=getty@tty1.service schon mit und übernimmt tty1 damit automatisch - sauberer läuft es trotzdem mit sudo systemctl disable getty@tty1.service, damit dort kein ungenutzter Login-Prompt mehr mitstartet.

Hinweis: An echter Hardware bestätigt: Ohne xserver-xorg-video-fbdev und die Xorg-Konfiguration oben bricht owlbox-kiosk.service sofort mit „no screens found“ ab - sichtbar aber nicht in journalctl -u owlbox-kiosk (nur „status=1“), sondern im eigentlichen X-Server-Log unter /var/log/Xorg.0.log (dort: „open /dev/dri/card0: No such file or directory“).

Läuft doch eine volle Desktop-Umgebung (z.B. die volle „Legacy“-Variante statt Lite geflasht): alternativ über deren Autostart-Datei einhängen - ~/.config/lxsession/LXDE-pi/autostart um die Zeile @/opt/owlbox/scripts/kiosk.sh ergänzen.

10. Konfigurationsdatei (config.yaml)

Alle in dieser Anleitung genannten Werte (Pins, Schrittweiten, Zeiten) stehen gesammelt in config/config.yaml (aus config/config.example.yaml kopieren). Die wichtigsten Abschnitte:

Abschnitt	Wichtigste Werte
audio:	alsa_device, mixer_control, default_volume, volume_step, chime_enabled
rfid:	reader, sck_pin, mosi_pin, miso_pin, cs_pin, reset_pin, poll_interval
gpio:	button_next/prev, encoder_clk/dt/switch, backlight_pin, brightness_encoder_clk/dt, shutdown_hold_seconds, seek_hold_seconds
playback:	restart_track_after_seconds, position_save_interval, auto_sleep_minutes, sleep_fade_seconds
network:	hotspot_ssid, hotspot_password, hotspot_after_seconds
web:	host, port, secret_key

Hinweis: config/config.yaml ist in .gitignore eingetragen, damit lokale, gerätespezifische Werte (insb. das Session-Secret) nie versehentlich committet werden.